

AI Usage Transcript — COMS3011A Lab 1

Constraints given up front

- Use better-sqlite3 directly (no ORM), Next.js App Router only
- No DELETE route — tasks can only be archived, never deleted
- Overdue must be derived at read time, never stored as a column or 4th status
- Single SQLite table, no separate topic/status tables

Example 1: Correcting a broken build (JSX placement)

What happened: A form component was generated but placed after the `return()` statement had already closed, and outside the JSX tree entirely — causing a build failure.

My correction: I flagged the exact error and asked for the file to be rewritten with everything correctly inside the JSX tree.

Result: Full corrected file provided, build succeeded.

Example 2: Correcting a terminal/file confusion

What happened: A block of file content (meant for a `.ts` file) was pasted directly into the terminal, causing bash to interpret it as commands and fail with syntax errors, rather than being saved to the file.

My correction: I reported the exact terminal output showing the failure, and asked for clarification on how to correctly create the file.

Result: Clarified the distinction between terminal commands and file content, and used a heredoc / VS Code editing approach instead.

Example 3: Iterating on UI/UX behaviour

What happened: Several rounds of refining the sort and filter interaction (column-based sort buttons, filter dropdowns, highlight-on-select behaviour) did not match my intended UX on the first pass.

My correction: I described the exact discrepancy each time (e.g. “the blue highlight should only show when the sort is actually applied”) until the behaviour matched what I wanted.

Result: Final sort/filter implementation matching the intended design.

Example 4: Reviewing backend design decisions

What happened: The initial database schema and API design were proposed with overdue and archive status handled through explicit design choices (`archived_at` timestamp instead of deletion, overdue computed at read time).

My correction: I asked clarifying questions about these decisions and confirmed they matched the brief’s constraints (tasks cannot be deleted, overdue is not a selectable status) before proceeding to implementation.

Result: Schema and API implemented and tested against these constraints, confirmed via the test suite.

Collaboration and AI Declaration

Across this lab, the working pattern was consistent: I described the feature or requirement I needed (e.g. the database schema, the API routes, the edit-in-place UI, the sort/filter behaviour), the AI produced an initial implementation, and I then reviewed the result against what I actually wanted —

directing specific corrections to UX, interaction behaviour, and layout until it matched my intent. The same pattern applied to testing: I specified which four behaviours needed test coverage (task creation and display, status changes, overdue detection, and persistence across a restart), and the AI implemented the test cases against that specification, which I then ran and confirmed passing myself.

The application itself, as well as the preceding document, was built and reviewed with: Claude-Web[Claude Sonnet 5]